

Practical Implementation Sheet

Department: CSE-AI

Semester: 5

Academic Year: 2026-27

Division: C

Batch: 3

Practical No: 3

Roll no: 59

PRN No: 12413745

Name of Student: Netra Lalwani

Subject Name: Deep Learning

Problem Statement:

Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

1. Import Libraries

- Import TensorFlow/Keras, NumPy, Pandas, Matplotlib, and Scikit-learn libraries required for model building, evaluation, and visualization.

2. Load the Dataset

- Load the MNIST dataset.
- Separate the features (pixel values) and labels (digit classes).

3. Preprocess the Data

- Normalize pixel values to the range **[0, 1]**.
- Reshape images into one-dimensional vectors of size **784 (28 × 28)**.
- Split the data into training and testing sets.

4. Define the Neural Network

- Create a Sequential model.
- Add an input layer.
- Add one or more hidden layers with **ReLU** activation.

- Add an output layer with **Softmax** activation for multi-class classification.

5. Compile the Model

- Select the **Adam optimizer** with a specified learning rate.
- Use **Sparse Categorical Crossentropy** as the loss function.
- Use **Accuracy** as the evaluation metric.

6. Train the Model (Forward and Backpropagation)

- Perform forward propagation to generate predictions.
- Compute the loss between predicted and actual outputs.
- Perform backpropagation to compute gradients.
- Update weights using the optimizer.
- Repeat for the specified number of epochs.

7. Experiment with Different Learning Rates

- Train the model using different learning rates (e.g., **0.01, 0.001, 0.0001**).
- Record the performance metrics for each learning rate.

8. Experiment with Different Epochs

- Train the model using different numbers of epochs (e.g., **5, 10, 20**).
- Observe the effect on training and validation performance.

9. Evaluate the Model

- Predict the labels for the test dataset.
- Calculate:
 - Accuracy
 - Precision
 - Recall
 - F1-Score

- Generate a Confusion Matrix and Classification Report.

10. Visualize Results

- Plot Training Accuracy vs Validation Accuracy.
- Plot Training Loss vs Validation Loss.
- Plot Learning Rate vs Accuracy.
- Plot Epochs vs Accuracy.
- Plot Precision, Recall, and F1-score comparison.
- Plot ROC Curve (optional for advanced analysis).

11. Analyze Performance

- Compare the results obtained with different learning rates and epochs.
- Identify the combination that provides the highest accuracy and best convergence with minimum loss.

12. Display Final Results

- Print the performance comparison table.
- Display the best-performing model configuration and evaluation metrics.

Code:

```
!pip install kagglehub -q

import kagglehub
import numpy as np
import os, struct, time
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.metrics import confusion_matrix, classification_report

path = kagglehub.dataset_download("hojjatk/mnist-dataset")
print("Path to dataset files:", path)
```

```

all_files = []
for root, dirs, fnames in os.walk(path):
    for fn in fnames:
        all_files.append(os.path.join(root, fn))

print("Files found:")
for f in all_files:
    print(" ", f)

def find_file(keyword):
    candidates = [f for f in all_files if keyword in os.path.basename(f).lower()]
    if not candidates:
        raise FileNotFoundError(f"No file matched keyword: {keyword}")
    candidates.sort(key=len) # prefer shortest path -> avoids nested duplicates
    return candidates[0]

def read_idx_images(filename):
    with open(filename, 'rb') as f:
        magic, num, rows, cols = struct.unpack(">IIII", f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8).reshape(num, rows, cols)

def read_idx_labels(filename):
    with open(filename, 'rb') as f:
        magic, num = struct.unpack(">II", f.read(8))
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return data

train_images_path = find_file("train-images")
train_labels_path = find_file("train-labels")
test_images_path = find_file("t10k-images")
test_labels_path = find_file("t10k-labels")

print("\nUsing:")
print(" train images:", train_images_path)
print(" train labels:", train_labels_path)
print(" test images :", test_images_path)
print(" test labels :", test_labels_path)

x_train = read_idx_images(train_images_path)
y_train = read_idx_labels(train_labels_path)
x_test = read_idx_images(test_images_path)

print("\nTrain:", x_train.shape, y_train.shape)
print("Test :", x_test.shape, y_test.shape)

```

```

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train_flat = x_train.reshape(-1, 28*28)
x_test_flat = x_test.reshape(-1, 28*28)

print("Flattened train shape:", x_train_flat.shape)
print("Flattened test shape :", x_test_flat.shape)

fig, axes = plt.subplots(2, 8, figsize=(14, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(x_train[i], cmap="gray")
    ax.set_title(f"Label: {y_train[i]}")
    ax.axis("off")
plt.suptitle("Sample MNIST digits")
plt.tight_layout()
plt.show()

# Class distribution
plt.figure(figsize=(7,4))
sns.countplot(x=y_train)
plt.title("Training Set Class Distribution")
plt.xlabel("Digit"); plt.ylabel("Count")
plt.show()

def build_model(lr=0.001):
    model = keras.Sequential([
        layers.Input(shape=(784,)),
        layers.Dense(128, activation="relu", name="hidden1"),
        layers.Dense(64, activation="relu", name="hidden2"),
        layers.Dense(10, activation="softmax", name="output")
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=lr),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"]
    )
    return model

model = build_model(lr=0.001)
model.summary()

```

```

tf.keras.backend.clear_session()
manual_model = build_model(lr=0.001)
optimizer = keras.optimizers.Adam(learning_rate=0.001)
loss_fn = keras.losses.SparseCategoricalCrossentropy()

@tf.function
def train_step(x_batch, y_batch):
    with tf.GradientTape() as tape:
        # ---- FORWARD PASS ----
        predictions = manual_model(x_batch, training=True)
        loss = loss_fn(y_batch, predictions)
        # ---- BACKWARD PASS (backpropagation) ----
        gradients = tape.gradient(loss, manual_model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, manual_model.trainable_variables))
    acc = tf.reduce_mean(
        tf.cast(tf.equal(tf.argmax(predictions, axis=1),
                        tf.cast(y_batch, tf.int64)), tf.float32))
    return loss, acc, gradients

batch_size = 128
dataset = tf.data.Dataset.from_tensor_slices((x_train_flat, y_train))
dataset = dataset.shuffle(10000).batch(batch_size)

manual_losses, manual_accs, grad_norms = [], [], []
print("Manual training loop (3 epochs) - explicit forward + backprop:")
for epoch in range(3):
    epoch_loss, epoch_acc, n_batches = 0, 0, 0
    for x_batch, y_batch in dataset:
        loss, acc, grads = train_step(x_batch, y_batch)
        epoch_loss += loss.numpy(); epoch_acc += acc.numpy(); n_batches += 1
    grad_norm = tf.linalg.global_norm(grads).numpy()
    manual_losses.append(epoch_loss / n_batches)
    manual_accs.append(epoch_acc / n_batches)
    grad_norms.append(grad_norm)
    print(f"Epoch {epoch+1}: loss={manual_losses[-1]:.4f}, "
          f"acc={manual_accs[-1]:.4f}, grad_norm={grad_norm:.4f}")

plt.figure(figsize=(6,4))
plt.plot(range(1, len(grad_norms)+1), grad_norms, marker="o", color="crimson")
plt.title("Gradient Norm per Epoch (Backprop Signal Strength)")
plt.xlabel("Epoch"); plt.ylabel("Global Gradient Norm")
plt.grid(alpha=0.3)
plt.show()

```

```

learning_rates = [0.1, 0.01, 0.001, 0.0001]
epoch_options  = [5, 15, 30]

results = []
histories = {}

for lr in learning_rates:
    for ep in epoch_options:
        print(f"Training: lr={lr}, epochs={ep}")
        tf.keras.backend.clear_session()
        m = build_model(lr=lr)
        start = time.time()
        h = m.fit(x_train_flat, y_train,
                  validation_split=0.1,
                  epochs=ep, batch_size=256, verbose=0)
        elapsed = time.time() - start
        test_loss, test_acc = m.evaluate(x_test_flat, y_test, verbose=0)
        results.append({
            "lr": lr, "epochs": ep,
            "train_acc": h.history["accuracy"][-1],

            "val_acc": h.history["val_accuracy"][-1],
            "test_acc": test_acc,
            "test_loss": test_loss,
            "time_sec": elapsed
        })
        histories[(lr, ep)] = h.history

df_results = pd.DataFrame(results)
print(df_results)

pivot_acc = df_results.pivot(index="lr", columns="epochs", values="test_acc")
plt.figure(figsize=(7,5))
sns.heatmap(pivot_acc, annot=True, fmt=".3f", cmap="viridis")
plt.title("Test Accuracy: Learning Rate vs Epochs")
plt.xlabel("Epochs"); plt.ylabel("Learning Rate")
plt.show()

pivot_loss = df_results.pivot(index="lr", columns="epochs", values="test_loss")
plt.figure(figsize=(7,5))
sns.heatmap(pivot_loss, annot=True, fmt=".3f", cmap="rocket_r")
plt.title("Test Loss: Learning Rate vs Epochs")
plt.xlabel("Epochs"); plt.ylabel("Learning Rate")
plt.show()

```



```

plt.figure(figsize=(8,5))
plt.plot(histories[(0.01, 15)]["val_accuracy"], label="fixed lr=0.01")
plt.plot(h_sched.history["val_accuracy"], label="exponential decay lr (start=0.01)")
plt.title("Fixed vs Decaying Learning Rate")
plt.xlabel("Epoch"); plt.ylabel("Validation Accuracy")
plt.legend(); plt.grid(alpha=0.3)
plt.show()

best_row = df_results.loc[df_results["test_acc"].idxmax()]
print("Best config:\n", best_row)

tf.keras.backend.clear_session()
best_model = build_model(lr=best_row["lr"])
best_model.fit(x_train_flat, y_train, epochs=int(best_row["epochs"]),
               batch_size=256, verbose=0)

preds = np.argmax(best_model.predict(x_test_flat, verbose=0), axis=1)

cm = confusion_matrix(y_test, preds)
plt.figure(figsize=(8,7))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title(f"Confusion Matrix (lr={best_row['lr']}, epochs={int(best_row['epochs'])})")
plt.xlabel("Predicted"); plt.ylabel("Actual")
plt.show()

print(classification_report(y_test, preds))

wrong_idx = np.where(preds != y_test)[0][:16]
fig, axes = plt.subplots(2, 8, figsize=(14,4))
for ax, idx in zip(axes.flat, wrong_idx):
    ax.imshow(x_test[idx], cmap="gray")
    ax.set_title(f"T:{y_test[idx]} P:{preds[idx]}")
    ax.axis("off")
plt.suptitle("Misclassified Examples")
plt.tight_layout()
plt.show()

# -----
W1 = best_model.layers[0].get_weights()[0] # shape (784, 128)
fig, axes = plt.subplots(4, 8, figsize=(14, 7))
for i, ax in enumerate(axes.flat):
    ax.imshow(W1[:, i].reshape(28, 28), cmap="seismic")
    ax.axis("off")
plt.suptitle("Learned Weight Patterns – First Hidden Layer (32 of 128 neurons)")
plt.tight_layout()
plt.show()

```

```
summary = df_results.sort_values("test_acc", ascending=False).reset_index(drop=True)
print("Ranked results (best to worst):")
print(summary.to_string(index=False))
```

Output:

Using Colab cache for faster access to the 'mnist-dataset' dataset.

Path to dataset files: /kaggle/input/mnist-dataset

Files found:

```
/kaggle/input/mnist-dataset/train-images.idx3-ubyte
/kaggle/input/mnist-dataset/t10k-labels.idx1-ubyte
/kaggle/input/mnist-dataset/t10k-images.idx3-ubyte
/kaggle/input/mnist-dataset/train-labels.idx1-ubyte
/kaggle/input/mnist-dataset/t10k-labels-idx1-ubyte/t10k-labels-idx1-ubyte
/kaggle/input/mnist-dataset/t10k-images-idx3-ubyte/t10k-images-idx3-ubyte
/kaggle/input/mnist-dataset/train-labels-idx1-ubyte/train-labels-idx1-ubyte
/kaggle/input/mnist-dataset/train-images-idx3-ubyte/train-images-idx3-ubyte
```

Using:

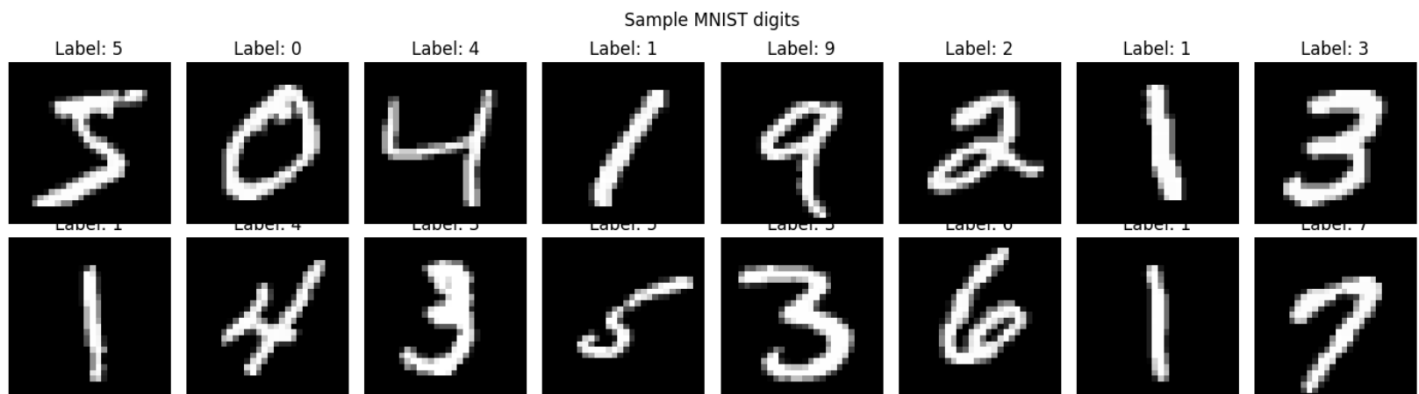
```
train images: /kaggle/input/mnist-dataset/train-images.idx3-ubyte
train labels: /kaggle/input/mnist-dataset/train-labels.idx1-ubyte
test images : /kaggle/input/mnist-dataset/t10k-images.idx3-ubyte
test labels : /kaggle/input/mnist-dataset/t10k-labels.idx1-ubyte
```

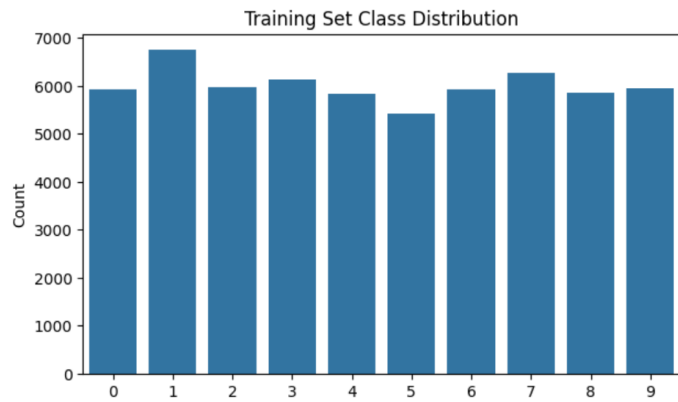
Train: (60000, 28, 28) (60000,)

Test : (10000, 28, 28) (10000,)

Flattened train shape: (60000, 784)

Flattened test shape : (10000, 784)





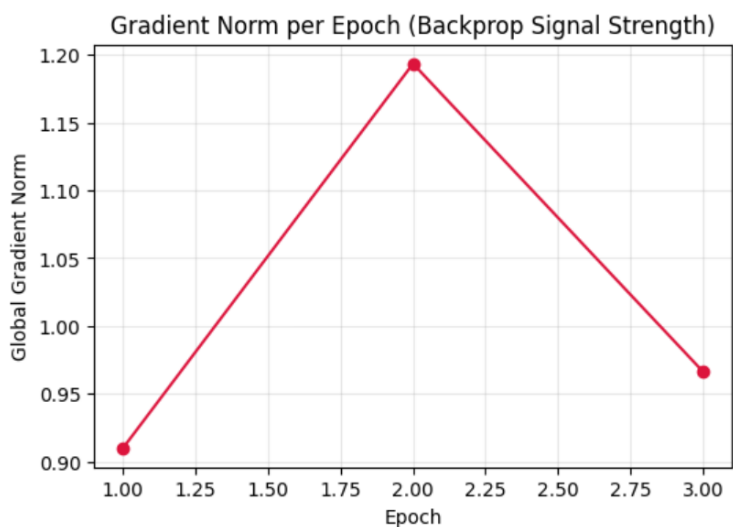
Model: "sequential"

Layer (type)	Output Shape	Param #
hidden1 (Dense)	(None, 128)	100,480
hidden2 (Dense)	(None, 64)	8,256
output (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

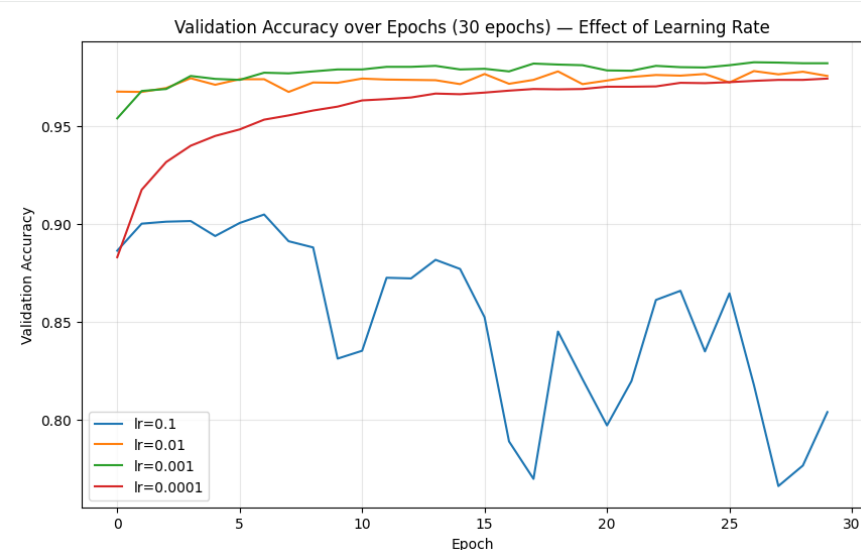
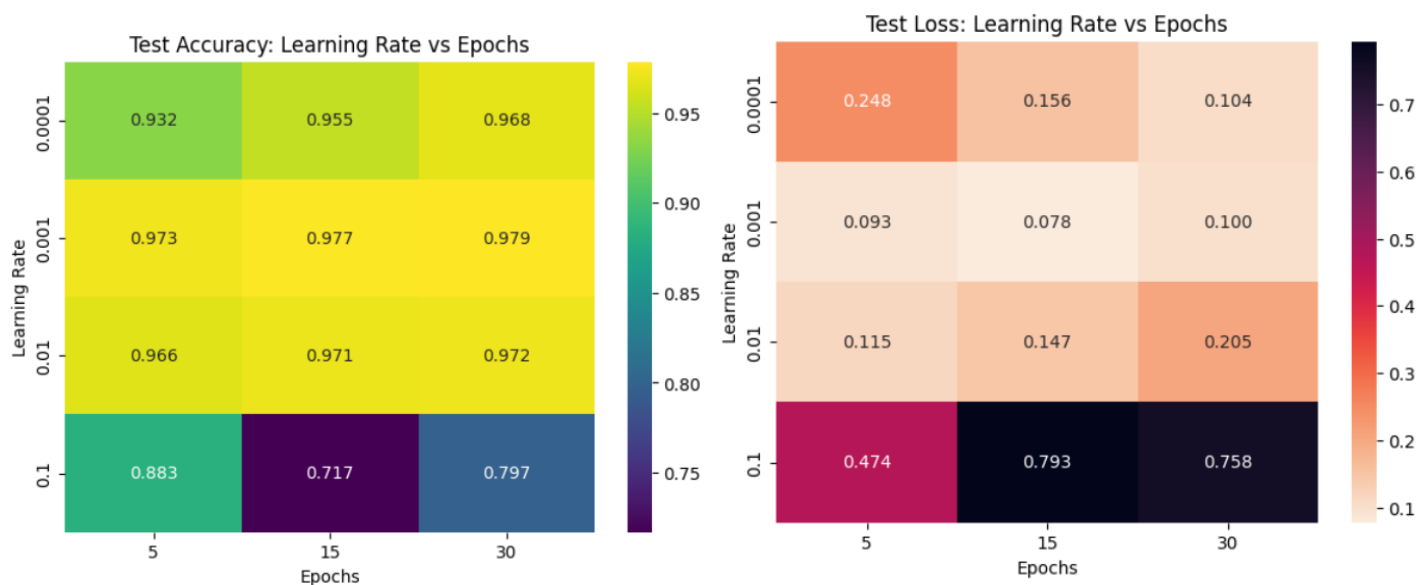
Manual training loop (3 epochs) – explicit forward + backward:

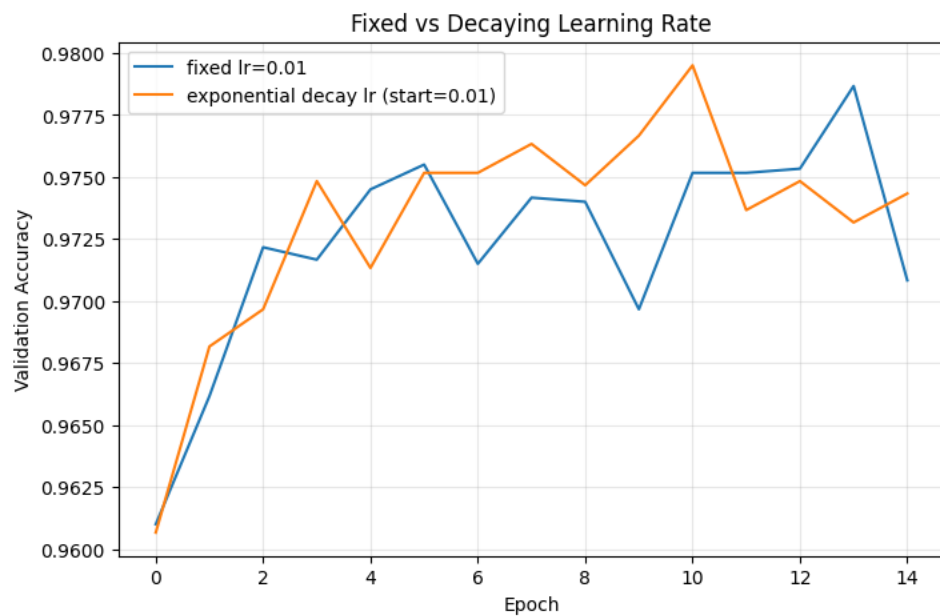
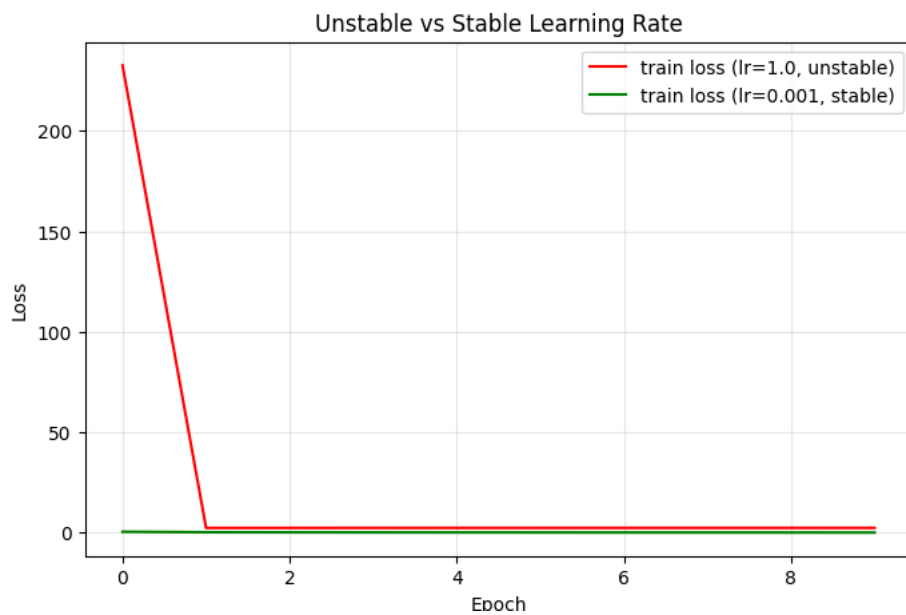
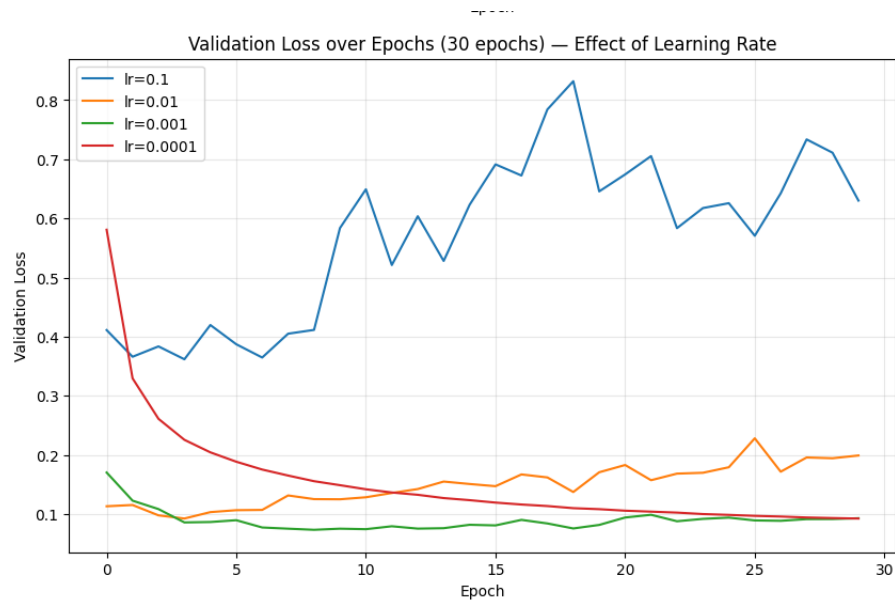
Epoch 1: loss=0.3328, acc=0.9033, grad_norm=0.9102
Epoch 2: loss=0.1404, acc=0.9584, grad_norm=1.1933
Epoch 3: loss=0.0982, acc=0.9704, grad_norm=0.9665



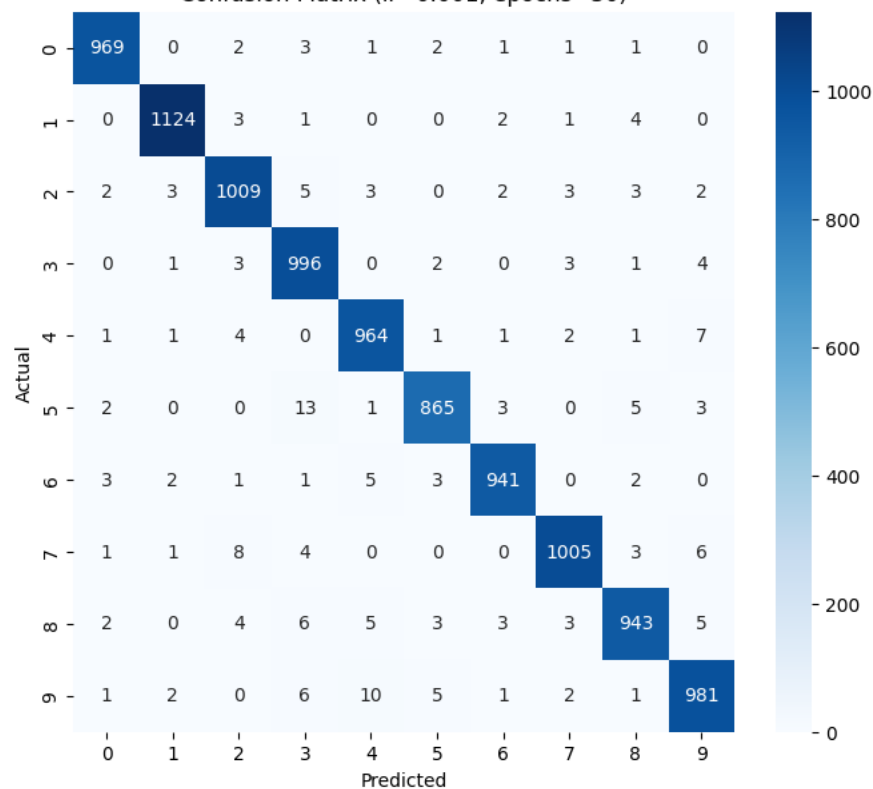
Training: lr=0.1, epochs=5
Training: lr=0.1, epochs=15
Training: lr=0.1, epochs=30
Training: lr=0.01, epochs=5
Training: lr=0.01, epochs=15
Training: lr=0.01, epochs=30
Training: lr=0.001, epochs=5
Training: lr=0.001, epochs=15
Training: lr=0.001, epochs=30
Training: lr=0.0001, epochs=5
Training: lr=0.0001, epochs=15
Training: lr=0.0001, epochs=30

	lr	epochs	train_acc	val_acc	test_acc	test_loss	time_sec
0	0.1000	5	0.888926	0.898667	0.8827	0.473677	8.945969
1	0.1000	15	0.715519	0.730167	0.7169	0.792891	26.764121
2	0.1000	30	0.779296	0.804000	0.7967	0.758383	42.548962
3	0.0100	5	0.979074	0.969500	0.9660	0.114914	8.765382
4	0.0100	15	0.989333	0.970833	0.9711	0.147390	22.025200
5	0.0100	30	0.993074	0.975833	0.9724	0.204727	42.084048
6	0.0010	5	0.976870	0.975667	0.9725	0.093026	8.714515
7	0.0010	15	0.996796	0.979167	0.9774	0.078249	24.404883
8	0.0010	30	1.000000	0.982333	0.9786	0.099853	50.186049
9	0.0001	5	0.925704	0.943000	0.9317	0.248494	8.119162
10	0.0001	15	0.956926	0.962167	0.9548	0.155563	21.428975
11	0.0001	30	0.977241	0.974500	0.9683	0.104143	44.252723



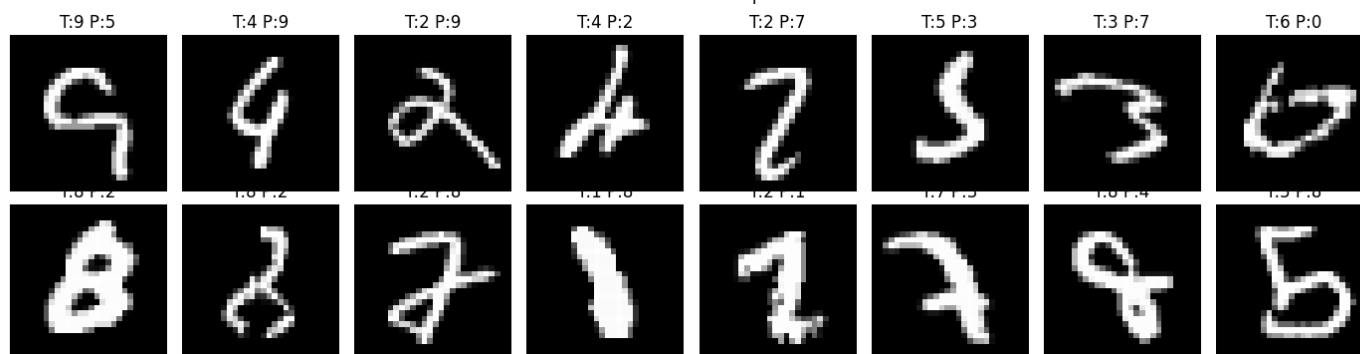


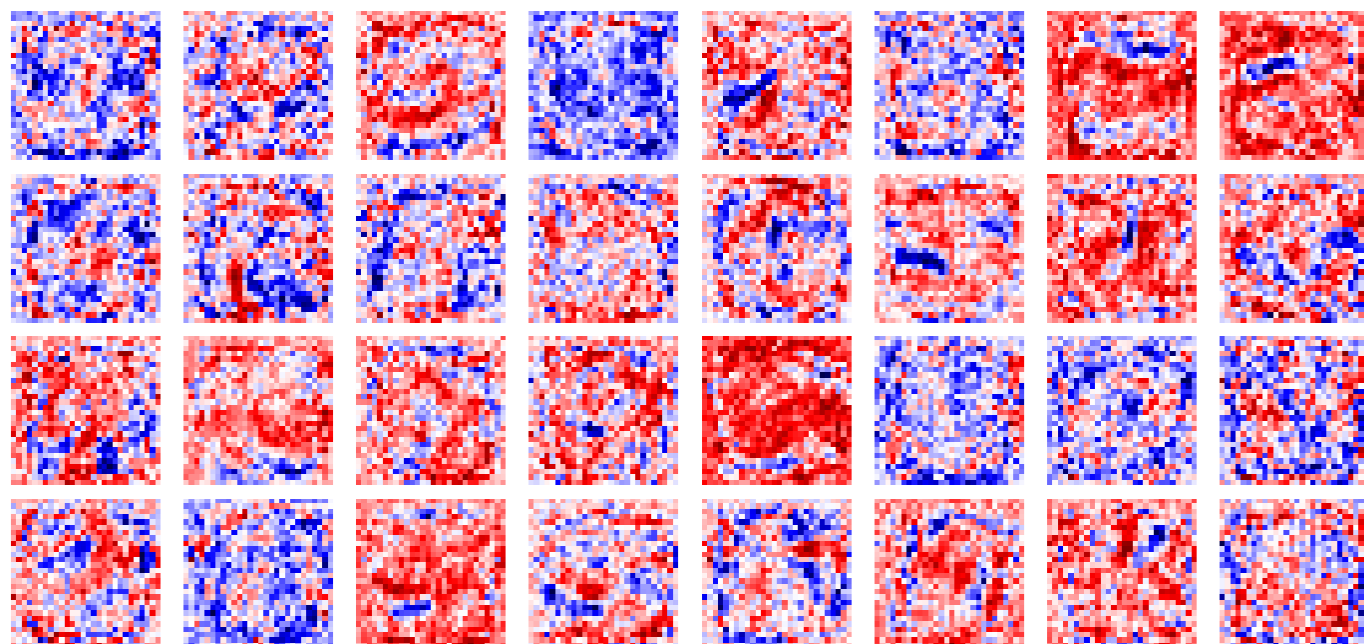
Confusion Matrix (lr=0.001, epochs=30)



	precision	recall	f1-score	support
0	0.99	0.99	0.99	980
1	0.99	0.99	0.99	1135
2	0.98	0.98	0.98	1032
3	0.96	0.99	0.97	1010
4	0.97	0.98	0.98	982
5	0.98	0.97	0.98	892
6	0.99	0.98	0.98	958
7	0.99	0.98	0.98	1028
8	0.98	0.97	0.97	974
9	0.97	0.97	0.97	1009
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

Misclassified Examples





Ranked results (best to worst):

lr	epochs	train_acc	val_acc	test_acc	test_loss	time_sec
0.0010	30	1.000000	0.982333	0.9786	0.099853	50.186049
0.0010	15	0.996796	0.979167	0.9774	0.078249	24.404883
0.0010	5	0.976870	0.975667	0.9725	0.093026	8.714515
0.0100	30	0.993074	0.975833	0.9724	0.204727	42.084048
0.0100	15	0.989333	0.970833	0.9711	0.147390	22.025200
0.0001	30	0.977241	0.974500	0.9683	0.104143	44.252723
0.0100	5	0.979074	0.969500	0.9660	0.114914	8.765382
0.0001	15	0.956926	0.962167	0.9548	0.155563	21.428975
0.0001	5	0.925704	0.943000	0.9317	0.248494	8.119162
0.1000	5	0.888926	0.898667	0.8827	0.473677	8.945969
0.1000	30	0.779296	0.804000	0.7967	0.758383	42.548962
0.1000	15	0.715519	0.730167	0.7169	0.792891	26.764121